- `body()` : ده الأكثر استخداماً. بيفتش في الـ Data اللي مبعوتة في الـ `req.body` (زي الـ Forms أو الـ JSON).
- `param()` : بيفتش في الـ Route parameters (مثلاً `/products/:productId`). مفيد جداً عشان تتأكد إن الـ ID سليم Mongo ID هو فعلاً اللي مبعوت في الـ URL.
- `query()` : بيفتش في الـ Query strings (مثلاً `?page=1&sort=desc`).
- `cookie()` : بيفتش في الكوكيز اللي مبعوتة من المتصفح.
- `header()` : بيفتش في الـ HTTP Headers.
- `check()` : ده الـ "الجوكر"؛ بيفتش في كل اللي فات (body, query, params, cookies, headers) بالترتيب ده.

1. الـ `check` و الـ `body` (المفتشين)

إحنا بنستخدم `check` أو `body` (وهي حالة خاصة للـ body بس) عشان نحدد إحنا هنفتش على إيه وبأمارة إيه.

- **الـ Chain Logic:** المكتبة دي بتعتمد على نظام الـ Chaining. بتقوله "فتش على الإيميل، واتأكد إنه إيميل، وشيل منه المسافات".
- **الـ Custom Messages:** تقدر تحط رسالة لكل غلطة لوحدها عشان اليوزر يفهم هو غلط في إيه بالظبط.

---

2. الـ `validationResult` (القاضي)

ده اللي بيجمع "تقرير التفتيش". الـ `check` لوحدها مش بتوقف الـ Request، هي بس بتعلم على الـ Request لو فيه غلط. إنت بقى في الـ Controller لازم تنادي على `validationResult(req)` عشان تعرف فيه مصايب ولا الدنيا تمام.

---

3. التنفيذ العملي (The Anatomy)

أ- في الـ Route (تجهيز المفتشين):

JavaScript

```
const { body } = require('express-validator');

router.post('/signup',
  [
    body('email')
      .isEmail().withMessage('الإيميل ده مش مظبوط يا ريس')
      .normalizeEmail(), // Sanitization: بيخلي الإيميل lowercase ويشيل النقط الزيادة
    body('password', 'الباسورد لازم يكون 5 أرقام وحروف على الأقل') // رسالة عامة للكل
```

```
      .isLength({ min: 5 })
      .isAlphanumeric()
      .trim()
  ],
  authController.postSignup
);
```

ب- في الـ Controller (اتخاذ القرار):

JavaScript

```
const { validationResult } = require('express-validator');

exports.postSignup = (req, res, next) => {
  const errors = validationResult(req); // 1. اسحب تقرير الأخطاء

  if (!errors.isEmpty()) {
    // 2. لو فيه أخطاء، ارجع لليوزر وقوله إيه اللي باظ
    console.log(errors.array()); // Array فيه كل تفاصيل الغلط
    return res.status(422).render('auth/signup', {
      errorMessage: errors.array()[0].msg, // خد أول رسالة ووريها لليوزر
      oldInput: { email: req.body.email } // تريكة: رجعله الداتا اللي كتبها
عشان ميعيدش من الأول
    });
  }
  // 3. لو الدنيا تمام، كمل كود الـ Save والـ Hash
};
```

---

4. الـ Custom Validators (الليفل الوحش)

ساعات الـ `isEmail` مش كفاية. إنت عايز تتأكد مثلاً إن الإيميل مش موجود في الداتابيز أصلاً. هنا بنستخدم `custom()`.

JavaScript

```
body('email').custom((value, { req }) => {
  if (value === 'test@test.com') {
    throw new Error('الايميل ده ممنوع من العرض!');
  }
  return true; // لو الدنيا تمام لازم ترجع true
})
```

- الـ `custom` بتقبل **Async**؛ يعني تقدر تروح تعمل `User.findOne()` وتستنى الرد، ولو لقيت يوزر، تعمل `Promise.reject('Email exists')`.

```
<input
  class="<%= validationErrors.find(e => e.param === 'email') ? 'invalid' : ''
%>"
  type="email"
  name="email"
  value="<%= oldInputs.email %>"> ```

### تحليل الـ `errors` Object اللي بيطلع في الـ Console:
فيه ميثود اسمها Object هتلاقي طالع لك، `console.log(errors)` لما عملت
`array()`. الـ Array ده جواه Objects فيه كل واحد فيهم، تانية:
1. **`msg`**: الرسالة اللي إنت كتبتها في الـ Route.
2. **`param`**: اسم الـ Input اللي فيه المشكلة (مثلاً `email`).
3. **`value`**: القيمة اللي اليوزر بعتها وكانت غلط.
4. **`location`**: هو لقى الغلط ده فين (body, query, params).
```

1. الخطوة الأولى: الـ Route (وضع الفخ)

هنا بنحدد إيه الداتا اللي هنفتش عليها وإيه هي الشروط.

JavaScript

```
// routes/admin.js
const { body } = require('express-validator');

router.post(
  '/add-product',
  [
    body('title').isString().isLength({ min: 3 }).trim(),
    body('price').isNumeric()
  ],
  adminController.postAddProduct // لو الداتا عدت التفتيش أو لاء، هتروح هنا
);
```

---

2. الخطوة الثانية: الـ Controller (القاضي)

هنا بنشوف "تقرير التفتيش" وبنقرر: هنكمل ولا هنرجع اليوزر لبيته؟

JavaScript

```
// controllers/admin.js
const { validationResult } = require('express-validator');

exports.postAddProduct = (req, res, next) => {
  const { title, price } = req.body;
  const errors = validationResult(req); // بسحب تقرير الغلطات

  if (!errors.isEmpty()) {
    // لو فيه غلطات (يعني Array مش فاضي الغلطات)
    return res.status(422).render('admin/add-product', {
      pageTitle: 'Add Product',
      errorMessage: errors.array()[0].msg, // أول رسالة غلط
      oldInput: { title: title, price: price }, // رجع الداتا عشان متمسحش (UX)
      validationErrors: errors.array() // رجع الـ Array كله عشان نلون الـ
Inputs
    });
  }

  // لو الدنيا تمام.. كمل وسيف في الداتابيز
  const product = new Product({ title, price });
  product.save().then(() => res.redirect('/'));
};
```

---

3. الخطوة الثالثة: الـ EJS (المواجهة مع اليوزر)

هنا بقى السحر كله. الـ EJS لازم يهندل حالتين: أول مرة يفتح (الداتا فاضية) وتاني مرة لما يرجع بغلط (الداتا موجودة).

HTML

```
<form action="/admin/add-product" method="POST">
  <div class="form-control">
    <label for="title">Title</label>
    <input
      class="<%= validationErrors.find(e => e.param === 'title') ? 'invalid' :
'' %>"
      type="text"
      name="title"
      value="<%= oldInput.title %>">
      </div>

  <div class="form-control">
```

```
    <label for="price">Price</label>
    <input
      class="<%= validationErrors.find(e => e.param === 'price') ? 'invalid' :
'' %>"
      type="number"
      name="price"
      value="<%= oldInput.price %>">
  </div>

  <button type="submit">Add Product</button>
</form>
```

شرح "تشريح" الكود ده (The Breakdown):

1. `oldInput` : دي ميزة الـ **Persistence**. لو اليوزر كتب العنوان "آيفون" ونسي السعر، لما الصفحة ترجع، العنوان هيفضل مكتوب في الـ `value` . ده بيمنع الـ "User Frustration".
2. `validationErrors.find(...)` : بتاعه هو `param` دي حركة "صايعة". إحنا بنلف جوه الأخطاء، لو لقينا غلط الـ (اللي إنت غالباً مخليه بيلون الـ Border بالأحمر). بنضيف .. `title` اسمه CSS Class `invalid`
3. الـ `res.status(422)` : دي إشارة للمتصفح إن الـ Request وصل بس الداتا "معطوبة" منطقياً.

---

4. حتة فنية للـ Get Route

عشان الكود ميضربش Error أول مرة تفتح الصفحة (لأن `oldInput` و `validationErrors` مش هيكونوا موجودين)، لازم في الـ `getAddProduct` تبعت قيم ابتدائية فاضية:

JavaScript

```
exports.getAddProduct = (req, res, next) => {
  res.render('admin/add-product', {
    pageTitle: 'Add Product',
    errorMessage: null,
    oldInput: { title: '', price: '' }, // ميزعلش EJS قيم فاضية عشان الـــ
    validationErrors: []
  });
};
```